PolicyMe: AI-Powered Development Guide

Complete PRD for Building with AI Tools

Version: 1.0
Date: March 3, 2026
Author: Sirius (PolicyMe Development Team)
Purpose: Step-by-step guide for building PolicyMe using AI development tools

---

Executive Summary

This document provides a comprehensive, tool-specific Product Requirements Document (PRD) for building PolicyMe using modern AI development platforms. PolicyMe is an Enterprise B2B SaaS platform that transforms corporate policy documents into an interactive knowledge graph powered by GraphRAG AI[1][2].

Target Delivery: Functional MVP within 4-6 weeks using AI-assisted development tools with minimal hand-coding.

Recommended Tool Stack:

- **Frontend UI Generation:** v0.dev (Vercel) for initial component scaffolding
- **Full-Stack Development:** Bolt.new for rapid prototyping with deployment
- **Code Refinement:** Cursor AI for debugging and feature enhancement
- **Backend Logic:** Replit Agent for API endpoints and database integration
- **Design System:** Relume.io for wireframes and component library

---

Part 1: Understanding the AI Development Workflow

1.1 The Modern AI Development Pipeline

Building enterprise apps in 2026 follows a fundamentally different workflow than traditional coding[3][4]:

Traditional Development (Old Way):
Requirements → Design → Code → Test → Debug → Deploy
(8-12 weeks for MVP)

AI-Assisted Development (New Way):
Natural Language Prompt → AI Generates Code → Visual Refinement → Deploy
(2-4 weeks for MVP)

1.2 Tool Selection Matrix

Tool	Best For	PolicyMe Use Case
v0.dev	Component-level UI generation with React/Next.js	Generate Ask PolicyMe search interface, KPI cards, data tables
Bolt.new	Full-stack app with automatic deployment	Build complete Employee dashboard with routing
Cursor AI	Debugging and refining existing code	Fix GraphRAG integration bugs, optimize performance
Replit Agent	Backend APIs and database setup	Build document ingestion pipeline, Neo4j connections
Bubble	No-code full-stack with database	Alternative for non-technical builders (trade-off: less customization)

Table 1: AI tool comparison for PolicyMe development

1.3 The "Vibe Coding" Principle

Modern AI tools respond to natural language prompts. The quality of your output depends on prompt specificity[5]:

Bad Prompt (Generic):

> "Build a dashboard for policies"

Good Prompt (Specific):

> "Build a dark-mode dashboard with a left sidebar (240px, collapsible), glassmorphic KPI cards showing Total Policies and Active Conflicts, and a data table with columns: Policy Name, Type, Status, Risk Score. Use Next.js 14, Tailwind CSS, and shadcn/ui components. Background color: #0F111A."

Part 2: Phase-by-Phase AI Build Strategy

Phase 1: Design System & Wireframes (Week 1)

Tool: Relume.io
Goal: Generate sitemap and wireframe structure before coding

Step 1.1: Generate Marketing Site Wireframes

Relume Prompt:
Generate a B2B SaaS marketing site for PolicyMe, an AI-powered corporate policy management platform.

Target audience: Enterprise Compliance Officers, HR Directors, IT Admins at companies with 500+ employees.

Unique selling points:

- GraphRAG (Knowledge Graphs) for zero-hallucination AI answers
- Real-time policy conflict detection
- Visual citation transparency with micro-graphs
- Role-based dashboards (Employee, Analyst, Admin)

Site structure:

1. Home - Hero with 3D network animation, social proof (logos), feature highlights, "Request Demo" CTA
2. Platform - Product tour showing Macro-Graph Visualizer and Ask PolicyMe interface
3. Use Cases - Tabs for HR Compliance, IT Security, Legal Risk Management
4. Security - Trust badges (SOC 2, ISO 27001), data encryption details
5. Pricing - 3-tier pricing table (Starter $12K, Professional $36K, Enterprise custom)
6. Contact Sales - Form with calendly integration

Style: Enterprise-grade, high-tech aesthetic. Use Untitled UI component style.

Expected Output: Complete sitemap with 20-30 wireframe sections ready for Figma export.

Step 1.2: Generate Application Dashboard Wireframes

Relume Prompt:
Generate dashboard wireframes for PolicyMe, a B2B policy intelligence platform.

Create 3 distinct role-based dashboard layouts:

1. ANALYST VIEW (Command Center):

- Persistent left sidebar (240px, collapsible): Dashboard, Graph Visualizer, Conflicts, Reports, Settings
- Top section: 4 glassmorphic KPI cards (Total Policies, Active Conflicts, Compliance Score, Recent Queries)
- Center section: Large placeholder for custom network graph (label: "Macro-Graph - Custom D3.js Visualization")
- Bottom section: Data table "Conflicting Policies" with columns: ID, Policy Name, Conflict Type, Severity (chip), Affected Docs

2. EMPLOYEE VIEW (Minimalist Search):

- No sidebar, just top nav with logo and profile
- Center: Large search bar with placeholder "Ask PolicyMe anything..."
- Below search: Recent queries list (3 items with timestamps)
- Right panel (300px): Small placeholder box labeled "Micro-Graph - Citation Network"

3. ADMIN VIEW (Control Panel):

- Left sidebar same as Analyst
- Center: User management table with columns: Name, Email, Role (dropdown), Last Active, Status (Active/Pending chip)
- Top right: "Invite User" button (primary blue)
- Right panel: Document ingestion section with drag-drop file upload zone and processing progress bar

Use dark mode aesthetic (#0F111A background), modern enterprise SaaS style.

Expected Output: 3 dashboard layouts with component placeholders.

Step 1.3: Export to Figma

1. Click "Export to Figma" in Relume
2. Open in Figma, apply dark mode color tokens:
    - Background: `#0F111A`
    - Surface: `#1A1D2E`
    - Primary: `#4A90E2` (cyan blue)
    - Destructive: `#E74C3C` (red for conflicts)
    - Text: `#FFFFFF`
3. Add glassmorphism effects:
    - Background: `rgba(255, 255, 255, 0.05)`
    - Backdrop filter: `Blur(10px)`
    - Border: `1px solid rgba(255, 255, 255, 0.1)`

Phase 2: Frontend UI Components (Week 2)

Tool: v0.dev (Vercel)
Goal: Generate individual React components with Tailwind CSS

Step 2.1: Generate Ask PolicyMe Search Interface

v0.dev Prompt:
Create a React component for an AI search interface called "Ask PolicyMe".

Requirements:

- Centered layout with max-width 800px
- Large search input with placeholder "Ask PolicyMe anything..." (text-lg, rounded-xl)
- Auto-expanding textarea (minimum 3 rows, maximum 10 rows)
- Primary submit button on right side of input (cyan blue #4A90E2)
- Below input: Show last 3 recent queries as clickable pills with timestamps
- Dark mode styling: background #0F111A, input background #1A1D2E
- Use Tailwind CSS and shadcn/ui components

- TypeScript with proper types

On submit, show loading state with pulsing dots animation (400ms interval).

Export as standalone Next.js 14 component with App Router compatibility.

Expected Output: Complete `AskPolicyMe.tsx` component with styles.

Step 2.2: Generate Conflict Data Table

v0.dev Prompt:
Create a React component for a data table displaying policy conflicts.

Requirements:

- Table with floating pill-shaped rows (not traditional grid lines)
- Corner radius: 12px per row, 8px vertical gap between rows
- Hover state: Background shift to rgba(255,255,255,0.05) + 4px elevation shadow
- Columns:
    1. ID (monospace font, grey text)
    2. Policy Name (white text, bold)
    3. Conflict Type (categorical chip: red for "Contradiction", yellow for "Ambiguity")
    4. AI Confidence Score (horizontal gradient bar chart: green to red, 0-100%)
    5. Affected Documents (truncated list: "HR Policy, IT Security +2 more")
    6. Actions (eye icon for "View Details")
- Sortable columns (click header to sort)
- Dark mode: background #0F111A, row background #1A1D2E
- Use Tailwind CSS, shadcn/ui Table component
- TypeScript with ConflictRow interface

Sample data: 5 mock conflict rows with varied severity.

Expected Output: `ConflictTable.tsx` with sorting logic and mock data.

Step 2.3: Generate Glassmorphic KPI Cards

v0.dev Prompt:
Create a React component for 4 glassmorphic KPI metric cards.

Card specifications:

- Semi-transparent background: rgba(255, 255, 255, 0.05)
- Backdrop filter: blur(10px)
- Border: 1px solid rgba(255, 255, 255, 0.1)
- Padding: 24px, Border radius: 12px
- Shadow: 0 8px 32px rgba(0, 0, 0, 0.2)
- Grid layout: 4 columns, 16px gap, responsive (2 cols on tablet, 1 col on mobile)

4 KPI cards:

1. Total Policies: Large number (247) + trend indicator (+12 this month, green up arrow)
2. Compliance Score: 0-100 with color-coded ring chart (75 = yellow)
3. Active Conflicts: Red accent, large number (18) + pulsing animation
4. Recent Queries: Small list of 3 queries with "View all" link

Dark mode styling (#0F111A background).
Use Tailwind CSS, shadcn/ui Card component, Recharts for ring chart.
TypeScript with KPICard interface.

Expected Output: `KPIDashboard.tsx` with 4 interactive cards.

Step 2.4: Combine Components in v0.dev

v0.dev Prompt:
Create a complete Analyst Dashboard page combining previous components.

Layout:

- Left sidebar (240px fixed): Navigation with icons (Home, Graph, Conflicts, Reports, Settings)
- Top section: 4 KPI cards in grid (use previous KPIDashboard component)

- Center section: Heading "Active Conflicts" + ConflictTable component
- Right panel (300px fixed): Placeholder div "Graph Detail Panel"

Dark mode theme (#0F111A).
Use Next.js 14 App Router, Tailwind CSS.
Export as /app/dashboard/page.tsx.

Expected Output: Complete dashboard page with all components integrated.

Phase 3: Full-Stack App Scaffolding (Week 3)

Tool: Bolt.new
Goal: Build complete application with routing, API endpoints, and deployment

Step 3.1: Initialize PolicyMe App in Bolt.new

Bolt.new Prompt:
Build a Next.js 14 application for PolicyMe, an enterprise policy management platform.

Project structure:

- App Router with TypeScript
- Tailwind CSS + shadcn/ui components
- Dark mode theme (background: #0F111A, primary: #4A90E2)

Pages to create:

1. /app/page.tsx - Marketing landing page with hero, features, pricing
2. /app/dashboard/page.tsx - Analyst dashboard (import components from previous v0.dev work)
3. /app/ask/page.tsx - Employee search interface
4. /app/admin/page.tsx - Admin control panel
5. /app/api/query/route.ts - API endpoint for AI queries (return mock data for now)

Navigation:

- Marketing site: Top navbar with links (Home, Product, Pricing, Contact)
- App dashboards: Left sidebar with role-based navigation

Authentication:

- Add mock login page at /app/login with email/password form
- After login, redirect to /dashboard (no actual auth yet, just UI)

Deploy to Netlify with one-click deployment.

Expected Output: Full Next.js app with 5 pages, routing, and deployed URL.

Bolt.new Tips:

- If Bolt makes errors, don't restart. Paste the error message and say "Fix this error by [specific solution]"
- Bolt auto-installs packages. If it uses wrong versions, specify: "Use shadcn/ui v2.0 and Next.js 14.2"
- Download code after generation: Click "Download" → Continue in Cursor AI for refinements

Step 3.2: Add API Integration for GraphRAG

Bolt.new Follow-Up Prompt:
Add API integration for GraphRAG query processing.

Modify /app/api/query/route.ts:

- Accept POST request with body: { query: string, userId: string }
- Call external GraphRAG API endpoint: POST https://api.policyme.com/graphrag/query
  - Headers: Authorization: Bearer [API_KEY]
  - Body: { query, userId }
- Return response: { answer: string, citations: Citation[], microGraph: GraphData }
- Add error handling for API failures (return 500 with error message)

Modify /app/ask/page.tsx:

- On search submit, call /api/query endpoint

- Display AI answer in glassmorphic card below search bar
- Render citations as clickable pills
- Show micro-graph visualization using placeholder div (we'll add D3.js later)

Use environment variable GRAPHRAG_API_KEY from .env.local.
Add loading spinner during API call (use shadcn/ui Spinner component).

Expected Output: Working API integration with mock GraphRAG responses.

Phase 4: Backend & Database (Week 4)

Tool: Replit Agent
Goal: Build document ingestion pipeline and Neo4j database integration

Step 4.1: Create Document Upload API

Replit Agent Prompt:
Build a document upload and processing API for PolicyMe.

Requirements:

- Framework: Node.js with Express
- Endpoint: POST /api/documents/upload
- Accept file uploads: PDF, DOCX (max 50MB per file)
- Use Multer middleware for multipart/form-data handling
- Store files in AWS S3 bucket (use aws-sdk v3)
- After upload, trigger background job to parse document:
  - Extract text using pdf-parse for PDFs, mammoth for DOCX
  - Send text to Python NLP service for entity extraction (separate microservice)
  - Insert extracted entities into Neo4j graph database

Database setup:

- Neo4j AuraDB connection (use neo4j-driver)
- Schema: (:Policy {id, name, department, content})-[:CONTAINS]-> (:Clause {id, text, embedding})
- Create indexes on Policy.name and Clause.embedding

Background job processing:

- Use Bull queue with Redis for job queue
- Job: { documentId, s3Url, userId }
- Process jobs in parallel (max 5 concurrent)

Environment variables:

- AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, S3_BUCKET_NAME
- NEO4J_URI, NEO4J_USER, NEO4J_PASSWORD
- REDIS_URL

Return response: { documentId, status: "processing", estimatedTime: "3 minutes" }

Expected Output: Complete Express API with file upload, S3 storage, and Neo4j integration.

Replit Agent Tips:

- Choose "Full Build" mode (not Lite) for complete implementation
- If Agent asks questions, answer with specifics: "Use Bull v4, Neo4j driver v5"
- Test endpoints directly in Replit's built-in API tester

Step 4.2: Build GraphRAG Query Engine

Replit Agent Prompt:
Build a GraphRAG query processing microservice for PolicyMe.

Requirements:

- Framework: Python with FastAPI
- Endpoint: POST /graphrag/query
- Input: { query: string, userId: string }
- Process:
    1. Generate query embedding using OpenAI embeddings API (text-embedding-3-small model)
    2. Perform vector similarity search in Neo4j (find top 10 similar clauses)

    3. Traverse graph to retrieve related clauses (multi-hop: 2 levels deep)
    4. Construct context from retrieved clauses (max 4000 tokens)
    5. Send context + query to OpenAI GPT-4 API with system prompt:
       "You are PolicyMe, an AI assistant for corporate policy questions. Answer based only on provided context. Cite sources using [Policy Name § Clause ID] format."
    6. Parse GPT-4 response to extract answer text and citations
    7. Build micro-graph JSON: { nodes: [query, cited clauses], edges: [query→clause connections] }
- Return: { answer: string, citations: Citation[], microGraph: GraphData, confidence: float }

Dependencies:

- fastapi, uvicorn, openai, neo4j, numpy, langchain

Environment variables:

- OPENAI_API_KEY, NEO4J_URI, NEO4J_USER, NEO4J_PASSWORD

Performance targets:

- Vector search: <500ms
- Graph traversal: <1s
- Total response time: <8s

Add caching layer using Redis (cache query embeddings for 24 hours).

Expected Output: FastAPI microservice with GraphRAG pipeline.

Phase 5: Integration & Refinement (Week 5)

Tool: Cursor AI
Goal: Debug integration issues and optimize performance

Step 5.1: Import Bolt.new Code into Cursor

1. Download code from Bolt.new (ZIP file)
2. Open in Cursor AI: `cursor .` in project directory
3. Cursor will index codebase automatically

Step 5.2: Fix Common Integration Issues

Cursor AI Prompt:
I have a Next.js app that calls a FastAPI GraphRAG service. The frontend sends POST to /api/query, which should call the Python service at http://localhost:8000/graphrag/query.

Current error: "fetch failed - ECONNREFUSED"

Debug steps:

1. Check if FastAPI service is running (verify with curl test)
2. Fix CORS headers in FastAPI (allow origin: http://localhost:3000)
3. Update Next.js API route to handle fetch errors gracefully
4. Add timeout (10s) to fetch call with error message if exceeded

Show me the exact code changes needed in:

- /app/api/query/route.ts (Next.js API route)
- /backend/main.py (FastAPI CORS config)

Expected Output: Cursor will show side-by-side diff with fixes, explain changes, and allow you to accept/reject.

Step 5.3: Optimize Graph Visualization Performance

Cursor AI Prompt:
The macro-graph visualization in /app/dashboard/graph/page.tsx is slow when rendering 500+ nodes using D3.js force-directed layout.

Optimization needed:

1. Switch from SVG to Canvas rendering for nodes (use d3-force with custom canvas drawing)
2. Implement level-of-detail: show node labels only when zoomed in >150%
3. Add quadtree for efficient collision detection

4. Debounce zoom/pan events (250ms delay)
5. Lazy load graph data (initial load: 100 nodes, load more on zoom)

Benchmark target: 60fps with 1000 nodes.

Show me the refactored code for the Graph component.

Expected Output: Optimized D3.js code with Canvas rendering.

Phase 6: Deployment & Testing (Week 6)

Tool: Bolt.new + Vercel + Railway
Goal: Deploy frontend and backend to production

Step 6.1: Deploy Frontend to Vercel

Manual Steps:

1. Push code to GitHub repository
2. Connect GitHub repo to Vercel: vercel.com → "New Project"
3. Configure environment variables in Vercel dashboard:
   - `GRAPHRAG_API_URL=https://api.policyme.com`
   - `OPENAI_API_KEY=[your-key]`
4. Deploy: Vercel auto-deploys on git push

Expected Output: Live URL: `https://policyme.vercel.app`

Step 6.2: Deploy Backend to Railway

Manual Steps:

1. Create Railway account: railway.app
2. Create new project → Deploy from GitHub
3. Add services:
   - **Service 1:** Node.js Express API (document upload)
   - **Service 2:** Python FastAPI (GraphRAG engine)
   - **Service 3:** Redis (Bull queue)
   - **Service 4:** Neo4j (graph database - use Neo4j AuraDB external)
4. Configure environment variables in Railway dashboard
5. Deploy: Railway auto-deploys from git

Expected Output: Live API: `https://api.policyme.railway.app`

Step 6.3: End-to-End Testing

Test Scenarios:

1. Upload test policy document (PDF) → Verify appears in admin dashboard
2. Ask query as Employee: "What is the remote work policy?" → Verify AI answer with citations
3. View conflict table as Analyst → Verify red nodes highlight on graph click
4. Invite new user as Admin → Verify email sent and user can log in

---

Part 3: AI Tool Prompt Library

3.1 Reusable Prompts for Common Tasks

Generate Dark Mode Component (v0.dev):
Create a React component for [component name].
Requirements:

- Dark mode: background #0F111A, surface #1A1D2E, primary #4A90E2
- Use Tailwind CSS and shadcn/ui
- TypeScript with proper interfaces
- Responsive: desktop first, tablet/mobile adapt
- [Specific functionality]
  Export as Next.js 14 App Router compatible component.

Add API Endpoint (Replit Agent):
Build an API endpoint for [feature name].
Framework: [Node.js Express / Python FastAPI]
Endpoint: [METHOD] /api/[route]
Input: { [fields] }
Process: [detailed steps]
Output: { [response fields] }
Error handling: [specific cases]
Environment variables: [list]

Debug Error (Cursor AI):
I'm getting this error: [paste error message]
Context: [what you were trying to do]
Relevant files: [list file paths]

Debug steps needed:

1. [specific check 1]
2. [specific check 2]
   Show me exact code changes with explanations.

Part 4: Cost Breakdown & Timeline

4.1 Tool Costs

Tool	Cost	Notes
v0.dev	Free tier: 200 generations/month	Sufficient for MVP, Pro: $20/month for unlimited
Bolt.new	Free tier: 20 prompts/day	Pro: $20/month for unlimited + priority
Cursor AI	Free: 2000 completions, Pro: $20/month	Includes Claude Sonnet, GPT-4 access
Replit Agent	$25/month (Core plan)	Includes 1000 Agent prompts, hosting
OpenAI API	Pay-per-use: ~$50-100/month	GPT-4: $0.03/1K tokens, Embeddings: $0.0001/1K
Neo4j AuraDB	Free tier: 50K nodes	Pro: $65/month for 200K nodes
Vercel	Free tier: Unlimited hobby projects	Pro: $20/month for production
Railway	Free: $5 credit/month	Pay-as-you-go: ~$10-20/month for MVP
Total (MVP)	**~$150-200/month**	Can start with free tiers, upgrade as needed

Table 2: AI tool cost breakdown

4.2 Development Timeline

Phase	Duration	Deliverables
Week 1	5-7 hours	Relume wireframes, Figma design system, color tokens
Week 2	8-10 hours	v0.dev components (search, table, cards, dashboard pages)
Week 3	10-12 hours	Bolt.new full app, routing, mock APIs, deployed to staging
Week 4	12-15 hours	Replit Agent backend (upload API, GraphRAG engine, Neo4j)
Week 5	8-10 hours	Cursor AI debugging, performance optimization, integration fixes
Week 6	5-7 hours	Production deployment, end-to-end testing, demo recording
Total	**48-61 hours**	**Functional MVP ready for Infosys presentation**

Table 3: Week-by-week timeline

Part 5: Success Criteria & Demo Preparation

5.1 MVP Must-Have Features

For Infosys Presentation, your demo MUST show:

1. **Employee Flow:** Log in → Ask query ("What is the remote work policy?") → Receive AI answer with citations → See micro-graph
2. **Analyst Flow:** Log in → View dashboard with KPI cards → Click "Active Conflicts" → See conflict table with red nodes → Click conflict → View side-by-side comparison
3. **Admin Flow:** Log in → Upload policy document (PDF) → See processing status → Document appears in policy library → Invite new user with role assignment

5.2 Presentation Script for Mentors

Opening (30 seconds):

> "PolicyMe solves the critical problem of policy fragmentation in enterprises. While other teams are building consumer apps for individuals, we built a B2B SaaS Command Center for enterprise compliance teams. Here's how it works for three different users."

Demo (3 minutes):

1. Show Employee search interface: "This is what 5,000 employees at Bank of America would see. Just a Google-like search bar."
2. Type query, submit, show AI answer with citations and micro-graph: "Notice the citations prove exactly where this answer came from. No black-box AI."
3. Switch to Analyst view: "This is the Macro-Graph Command Center. See these red nodes? Those are policy conflicts our AI detected automatically."
4. Click conflict, show side-by-side comparison: "HR says USB drives are banned, but IT issues USB drives to new hires. Without PolicyMe, this wouldn't be caught until an audit."

Technical Deep-Dive (2 minutes):

> "We built this using GraphRAG, not basic keyword search. Every policy is a node, relationships are edges. When you query, the AI traverses the graph to find the mathematically shortest path to the answer. That's why we have zero hallucinations."

Closing (30 seconds):

> "This is production-ready for enterprise deployment. We're talking to three Fortune 500 companies who spend $2M annually on manual compliance audits. PolicyMe reduces that to $36K per year."

5.3 Common Mentor Questions & Answers

Q: "Why not just use ChatGPT?"
A: "ChatGPT hallucinates because it guesses. PolicyMe uses GraphRAG—it mathematically traverses a knowledge graph. If the answer isn't in the graph, it says 'I don't know' instead of making something up."

Q: "How do you handle data privacy?"
A: "Enterprise data never leaves the customer's instance. We offer on-premise deployment with Docker/Kubernetes. For cloud, we use AES-256 encryption and SOC 2 compliance."

Q: "What's your business model?"
A: "B2B SaaS subscription. Starter tier: $12K/year for 100 users. Professional: $36K/year for 500 users. Enterprise: Custom pricing for Fortune 500."

Q: "How is this different from SharePoint?"
A: "SharePoint is a file storage system. PolicyMe is an intelligence layer. SharePoint shows you where documents are; PolicyMe tells you what they mean and when they contradict each other."

Part 6: Post-MVP Enhancements

6.1 Features to Add After Infosys Demo

Priority 1 (Months 2-3):

- SSO integration (Okta, Azure AD, Google Workspace)
- Slack/Teams bot for Ask PolicyMe queries
- Export conflict reports as PDF with evidence
- Version control for policy documents (track changes over time)

Priority 2 (Months 4-6):

- Advanced graph filtering (by department, date range, document type)
- Real-time collaboration (multiple analysts viewing same graph simultaneously)
- Mobile app (iOS/Android) with simplified Employee interface
- Multi-language support (Spanish, French, German)

Priority 3 (Months 7-12):

- Agentic AI workflows (auto-resolve low-severity conflicts)
- HRIS integrations (BambooHR, Workday) for automatic user provisioning
- Compliance tool integrations (Drata, Vanta) for policy evidence export
- Custom NLP model training (fine-tune for industry-specific terminology)

Summary: Your 6-Week Action Plan

Week 1: Generate wireframes in Relume.io → Export to Figma → Apply dark mode theme

Week 2: Build UI components in v0.dev → Ask PolicyMe search, conflict table, KPI cards, dashboard layouts

Week 3: Create full app in Bolt.new → Combine components, add routing, deploy to Netlify

Week 4: Build backend in Replit Agent → Document upload API, GraphRAG query engine, Neo4j integration

Week 5: Debug in Cursor AI → Fix integration issues, optimize graph performance, add error handling

Week 6: Deploy to production → Vercel (frontend) + Railway (backend) → End-to-end testing → Record demo video

Presentation Day: Show 3-minute live demo → Explain GraphRAG technical advantage → Answer mentor questions confidently

---

References

[1] Memgraph. (2024). Prerequisites for GraphRAG systems. https://memgraph.com/docs/ai-ecosystem/graph-rag/prerequisites

[2] Infrasity. (2025). The ultimate PRD guide for your B2B SaaS success in 2025. https://www.infrasity.com/blog/b2b-saas-prd

[3] Zapier. (2023). The 9+ best no-code app builders in 2026. https://zapier.com/blog/best-no-code-app-builder/

[4] Lovable. (2025). 8 AI platforms for building apps in 2026 (Compared). https://lovable.dev/guides/top-ai-platforms-app-development-2026

[5] WeWeb. (2026). 10 best no code app builder platforms (2026 Guide). https://www.weweb.io/blog/best-no-code-app-builder-platforms